前置知識與目標

- 前置知識為遞迴、鏈結結構、堆疊與歸納證明。
- 能以精確術語描述樹，追蹤走訪、BST 更新與堆積操作，並維護各自的不變量。
- 能說明一般二元樹、二元搜尋樹及二元堆積的差異，而不只依外形判斷。

樹的數學定義與術語

- 無向樹是連通且無環的簡單圖。任意兩點之間恰有一條簡單路徑，含 `n` 點時有 `n-1` 邊。
- 選定 root（根）後，每個非根節點具有唯一 parent（父節點）。children（子節點）是距根多一條邊的鄰點。
- depth（深度）為根到節點的邊數；height（高度）為節點到最深後代葉的邊數。整棵樹高度是根高度。
- leaf（葉）沒有子節點。森林是互不相交的樹集合。空樹的高度約定為 −1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二元樹左右位置有語義

- 二元樹每個節點有至多一個左子節點及一個右子節點。只有左子與只有右子是不同結構。
- 高度 `h≥0` 的二元樹最多有 `2^{h+1}-1` 點，因第 `d` 層最多有 `2^d` 點。
- full binary tree 在本教材指每個內部節點恰有兩子。complete 指最後一層以外填滿，最後一層由左連續填入。
- perfect 指各葉同深且內部節點都有兩子。這些名稱在不同教材可能有變體，必須先固定定義。

有些教材將 full 用於所有層都填滿的 perfect tree，並另稱每個內部節點恰有兩子的樹為 strict。閱讀時以定義而非名稱判別。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

葉節點與分支數：先數邊，再數節點

非空二元樹以 n_0、n_1、n_2 分別表示 0、1、2 個子節點的節點數。每個非根節點恰對應一條父子邊，因此邊數既是 n−1，也是 n_1+2n_2。

代入 $n = n_0 + n_1 + n_2$，得到 $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$，消去 n_1 即得 **$n_0=n_2+1$**。這個等式不要求樹為完全樹或每個內部節點都有兩個孩子；空樹不符合此結論。

例：55 個節點、其中 22 個只有一子。由 $55 = n_0 + 22 + (n_0 - 1)$，得 $n_0=17$、$n_2=16$。對一般根樹，同樣的邊數法給出 $n_0 = 1 + \sum_{d\geq 2}(d-1)n_d$，求和範圍 d≥2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

高度界與完整樹的索引

本章高度以邊數計，根的 depth 為 0。深度 d 最多 2^d 點，歸納步是每個深度 d 節點至多生兩個孩子。加總得 $n \leq 2^{h+1} - 1$，故非空樹 $h \geq \lceil \log_2(n+1) \rceil - 1$；鏈狀樹達到最大高度 n−1。

若使用 root level=1、height=層數的慣例，上式的高度數字全都加 1。不要把兩種慣例混在同一次計算。

0 起始完整樹以 i 表示節點時，左子 2i+1、右子 2i+2、父節點 ⌊(i−1)/2⌋；子索引必須小於 n。n=10 時最後有子節點為 i=4，葉索引 5 至 9，共 5 個。1 起始版本的父子公式依序為 ⌊j/2⌋、2j、2j+1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

鏈結與陣列表示

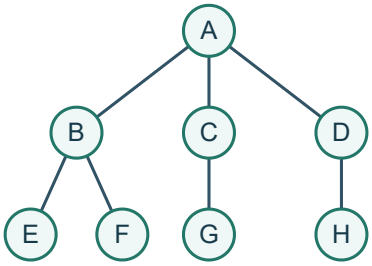
- 鏈結二元樹以 `{key,left,right}` 表示，可自然容納不規則形狀。
- 完全二元樹可放在 0 起始陣列，節點 `i` 的左右子索引是 `2i+1`、`2i+2`，父索引是 `⌊(i-1)/2⌋`，父式只對 `i>0` 使用。
- 高度 `h` 的單鏈若硬套完整位置編號，索引可達指數規模。陣列表示不是所有二元樹都省空間。
- 由索引算子節點時，要在乘法或加法前確保不溢位，並檢查是否小於元素數。

本章堆積採 0 起始索引；演算法課常採 1 起始，父、子公式分別為 ⌊j/2⌋、2j、2j+1。兩者表示同一結構，代入前先確認索引起點。

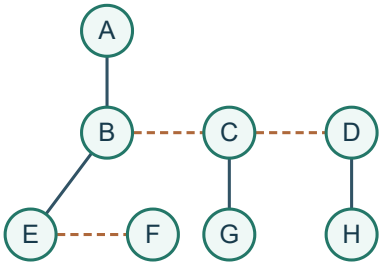
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左子右兄弟：保留有序一般樹

一般樹與左子右兄弟表示



原樹：A 的孩子依序為 B、C、D



實線 left = 第一個孩子

虛線 right = 下一個兄弟

右側的 C 是 B 的兄弟，不是 B 在原樹中的孩子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左子右兄弟的讀法與樹的計數

每個節點的 left 指向第一個孩子，right 指向下一個兄弟；沿第一個孩子的 right 鏈可依序列出原樹所有孩子。若要表示森林，根之間也可依序以 right 相連。這個對應保留的是 **有序** 根樹；忽略原本兄弟順序會失去一對一關係。

令 C_n 是 n 個無標記節點的有序二元樹形狀數，包含 $C_0=1$。根分配 i 個節點給左子樹，其餘 n−1−i 個給右子樹，故 $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$。逐項算得 $C_1=1$、$C_2=2$、$C_3=5$、$C_4=14$。

對固定的 n 個互異且全序的 BST 鍵，每一種二元樹形狀都只有一種符合中序遞增的標鍵方式，所以 BST 的形狀數同為 C_n。若節點標籤可任意排列且視為不同，一般二元樹則有 $n!C_n$ 種；不能混淆這兩個計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Catalan 閉式：用括號路徑計數

將空樹編碼為空字串；非空樹編為 (、左子樹編碼、)、右子樹編碼。匹配根的第一對括號，可唯一切回左右子樹，故 n 節點二元樹形狀與 n 對合法括號一一對應。

將左括號視為上步、右括號視為下步，全部上、下各 n 步有 $\binom{2n}{n}$ 種。對第一次跌到 −1 的路徑，反射到該位置以前的步驟，得到上步 n+1、下步 n−1 的路徑；反射可逆，故不合法路徑有 $\binom{2n}{n-1}$ 種。

合法數為 $C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}$。其中 n=0 得 1，表示空樹也計作一種形狀。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三種深度優先走訪

- preorder（前序）依「根、左、右」。inorder（中序）依「左、根、右」。postorder（後序）依「左、右、根」。
- 例樹根為 8，左子 3 的左右子為 1、6，右子 12 的左子為 10。
- 前序為 `8,3,1,6,12,10`。中序為 `1,3,6,8,10,12`。後序為 `1,6,3,10,12,8`。
- 對每個子樹按相同規則遞迴，歸納即可證明每個節點恰訪問一次，時間 `O(n)`。

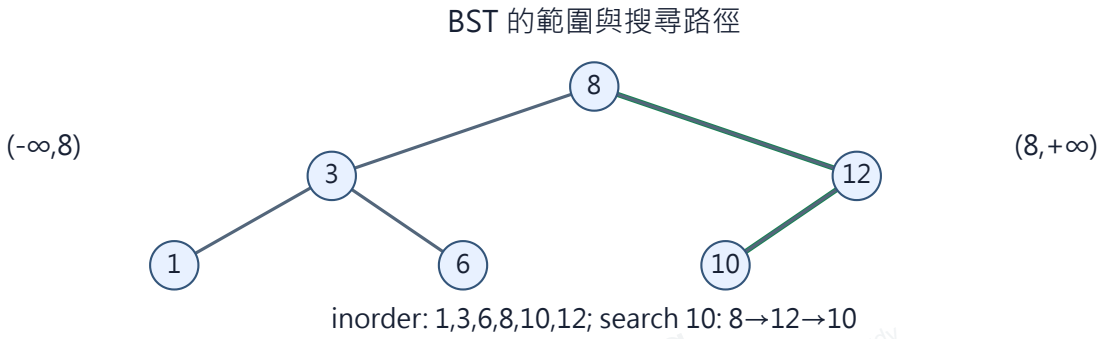
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴走訪與迭代中序的完整狀態

```
visit(x,order):
    if x==null: return
    if order==pre: emit(x)
    visit(x.left,order)
    if order==in: emit(x)
    visit(x.right,order)
    if order==post: emit(x)
inorder_iterative(root):
    stack=[]; x=root
    while x!=null or stack is not empty:
        while x!=null: stack.push(x); x=x.left
        x=stack.pop(); emit(x); x=x.right
```

https://github.com/xinhi6716/com

遞迴走訪與迭代中序的完整狀態 （續）



同一樹的中序堆疊底→頂與輸出：先push8,3,1得[8,3,1]；pop1輸出[1]；pop3輸出[1,3]後push6得[8,6]；pop6輸出[1,3,6]；pop8後進12的左鏈得[12,10]；pop10後pop12，結束為[1,3,6,8,10,12]。堆疊保存待輸出的祖先，每個節點push/pop恰一次，總Θ(n)，輔助O(h+1)。

一般二元樹的中序不保證排序；只有全子樹範圍成立的BST才有此性質。不能用本例的已排序結果替代BST表示不變量。

從前序與中序還原樹

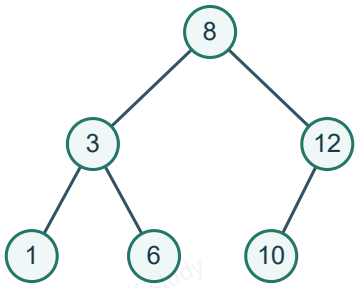
前序與中序如何切出子樹

pre: 8 | 3, 1, 6 | 12, 10
in: 1, 3, 6 | 8 | 10, 12

根 8 將中序切成左 3 個鍵、右 2 個鍵

左：pre 3,1,6；in 1,3,6

右：pre 12,10；in 10,12



重複套用：前序首鍵是根；中序決定左右子樹大小。

鍵必須互異；若輸入不相容，重建程序必須拒絕。

還原樹的正確性與限制

鍵互異且兩序列確實來自同一棵樹時，前序首項唯一決定根；根在中序的左、右段唯一決定兩子樹的鍵集合及大小。據此在前序切出同長區段，再對左右段遞迴。對節點數歸納，根及左右子樹均唯一，整棵樹因此唯一。

實作先建立 key→中序位置的映射，使用區間邊界與一個前序游標。每個鍵處理一次，期望 Θ(n) 時間、Θ(n) 映射空間與 O(h+1) 呼叫堆疊；雜湊表須另註最壞界。若鍵重複、集合不一致，或根的位置落在當前中序區間外，就拒絕輸入。

僅有前序與後序通常不唯一。單鍵 A 的唯一子節點 B 可放左或右，兩者的前序皆 A,B，後序皆 B,A。若另外保證每個內部節點恰有兩子且鍵互異，則可藉前序下一鍵在後序中定位左子樹，遞迴唯一還原。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

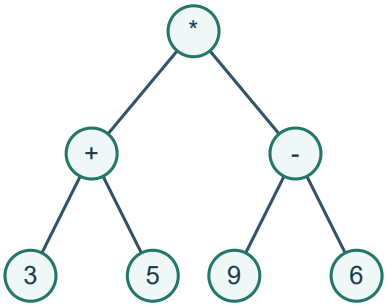
走訪的堆疊與佇列成本

- 非空樹遞迴 DFS 的額外堆疊 `O(h+1)`，以免 h=0 的單點例外被省略。退化鏈的 `h=n-1`，不能一律宣稱 `O(log n)`。
- 中序迭代法先把沿途左鏈 push，pop 一點並輸出後，再處理其右子樹。
- level-order（層序）以佇列逐層處理。原例結果是 `8,3,12,1,6,10`。
- 層序佇列需 `O(w)`，`w` 為最大層寬。平衡樹可能有 `w=Θ(n)`，與 DFS 堆疊不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

運算式樹與走訪順序

運算式樹：走訪順序與計算



中序 inorder： (3 + 5) × (9 − 6)

前序 preorder： × + 3 5 − 9 6

後序 postorder： 3 5 + 9 6 − ×

後序求值：8 × 3 = 24

中序要依運算子的優先順序及結合性加括號，才能保留原樹的語意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

運算式樹如何保留語意

運算子節點的左右孩子分別是左、右運算元。後序會先取得兩個運算元，才執行父運算子；例如後序 `3 5 + 9 6 − ×` 依序計算 8、3、24。二元減法與除法不可交換：先彈出的是右運算元，後彈出的是左運算元。

前序對應 prefix 表示法，後序對應 postfix。中序若直接省略括號，可能改變運算順序：樹 `(3+5)×(9−6)` 若寫成 `3+5×9−6` 就不是相同運算式。輸出中序時，要保留足夠括號，並考量運算子的優先順序及結合性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 不變量與搜尋

- 本章鍵互異。對任一節點 x，左子樹所有鍵 `<x.key`，右子樹所有鍵 `>x.key`。
- 搜尋 k 時，依比較結果丟棄根與其中一個子樹。被丟棄子樹不含 k，理由來自全子樹不變量。
- 上述例樹搜尋 10 的路徑為 `8→12→10`。搜尋 11 最後落到 10 的空右子位置。
- 搜尋、插入與刪除時間均為 `O(h+1)`。沒有平衡機制時，依序插入遞增鍵可使高度線性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 插入與重複鍵政策

- 沿搜尋路徑找到空子位置，插入新葉即可保留所有祖先的鍵範圍限制。
- 在原例插入 11，成為 10 的右子。只需檢查沿路界線 `8<11<12`、`10<11`。
- 若允許重複，不能一邊說嚴格不等式，一邊把相同鍵放任意子樹。可採每鍵計數、複合唯一鍵或明訂非嚴格規則。
- 實務容器若儲存 key-value，更新既有 key 的 value 與新增節點是不同操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 查找、插入與全域範圍驗證

```
find(root,k):
    x=root
    while x!=null:
        if k==x.key: return x
        if k<x.key: x=x.left
        else: x=x.right
    return not_found
insert(x,k,v):
    if x==null: return Node(k,v,left=null,right=null)
    if k<x.key: x.left=insert(x.left,k,v)
    else if k>x.key: x.right=insert(x.right,k,v)
    else: x.value=v
    return x
valid(x,lower,upper):
    if x==null: return true
    return lower<x.key<upper and valid(x.left,lower,x.key)            and
valid(x.right,x.key,upper)
root=insert(root,k,v)
```


BST 查找、插入與全域範圍驗證 （續）

此介面對同鍵更新 value，不配置第二個節點。valid 的上下界是可選的數學界，起始為 −∞、+∞；若機器鍵域已包含極值，實作要用「沒有界」的標記，不能拿最小／最大可表示鍵冒充無窮。

原樹查11的比較為8<11、12>11、10<11，進10.right的空位置而失敗；插11沿相同路徑配置葉，逐層接回10.right、12.left、8.right，外部也接回 root。已知的祖先區間依序為(8,+∞)、(8,12)、(10,12)，故11滿足全部限制。再次插11只更新該節點 value，節點數不變。

反例根8、左子3、3右子9：每條父子邊的局部大小都對，但 valid 在9處帶著(3,8)而拒絕。搜尋正確性同樣使用祖先區間；若 k<x.key，整棵右子樹的鍵都大於 x.key，不能含 k。查找與插入走一條高 h 的路徑，最壞 O(h+1)；遞迴插入另用 O(h+1) 堆疊。全域驗證必看全部 n 個節點，時間 Θ(n)，不是 O(h)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 刪除的三種情況

1. 葉節點直接從父連結移除。
2. 單一子節點以該子樹取代被刪位置。
3. 兩子節點以右子樹最小鍵，即中序後繼，取代邏輯內容，再刪除原後繼位置。

後繼不可能有左子，因此最後只需處理前兩種。若節點身分被外部 handle 使用，搬值與搬節點必須選擇一致語義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩子刪除的原創追蹤

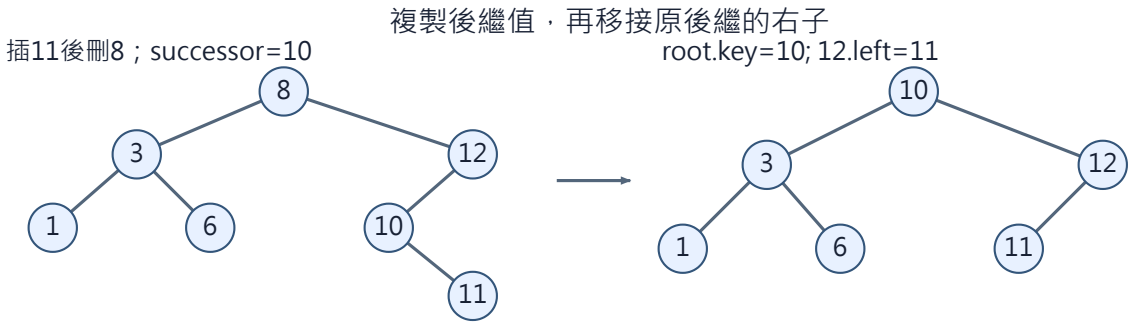
- 從原樹刪除 8，右子樹最小鍵是 10。令根鍵為 10，再刪除 12 的左子 10。
- 新中序為 `1,3,6,10,12`，恰是原有序序列移除 8。
- 後繼大於原根左子樹所有鍵，且不大於右子樹剩餘鍵，因此替換不破壞 BST 次序。
- 只檢查父與子大小並不足以驗證 BST。例如根 8 的左子 3 若有右後代 9，局部邊比較成立，整體仍違規。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST 刪除的完整遞迴與身分契約

```
erase(x,k):
    if x==null: return null
    if k<x.key: x.left=erase(x.left,k)
    else if k>x.key: x.right=erase(x.right,k)
    else:
        if x.left==null: y=x.right; release(x); return y
        if x.right==null: y=x.left; release(x); return y
        s=x.right
        while s.left!=null: s=s.left
        x.key_and_value=copy(s.key_and_value)
        x.right=erase(x.right,s.key)
    return x
root=erase(root,k)
```


BST 刪除的完整遞迴與身分契約 （續）



先在10右插11。刪8時後繼10沒有左子，但有右子11；複製10到原根後，遞迴刪原10位置，回傳11給12.left。根的左右範圍仍分別小於、大於10，中序恰少8。若刪根而只在函式內改區域指標，外部root可能指向已釋放節點，所以呼叫端必須接回回傳根。

此版承諾key-value集合，不承諾節點handle身分。若要求所有未刪元素的handle穩定，應以節點transplant更新parent及root，而不是複製後繼內容。泛型payload複製例外也需另定提交政策。成本是搜尋加後繼左鏈，共O(h+1)，無平衡時可Θ(n)。

一般後繼的單次成本與整段走訪

```
successor(x):
    require x is a live node and parent links are maintained
    if x.right!=null:
        x=x.right
        while x.left!=null: x=x.left
        return x
    p=x.parent
    while p!=null and x==p.right: x=p; p=p.parent
    return p
```

後繼是嚴格大於目前鍵的最小鍵，不是比較數值加1。若有右子樹，該子樹最左點是答案；否則沿父鏈跳過「剛走完其右子樹」的祖先，第一個把目前子樹放左側的祖先才是答案；跳到根外表示目前是最大鍵。此頁採普通child指標加parent，與本章的thread標記是不同表示。

平衡例根11、左5(2,7)、右17(13,19)：successor(5)沿右子到7；successor(7)先由7回5，再由5回11，答案11；successor(19)經17、11後回null。即使給了節點handle，一次successor仍可能走高度h，最壞O(h+1)，沒有自動的O(1)界。

從最左鍵開始反覆successor，輸出2,5,7,11,13,17,19。每條樹邊向下至多一次、向上至多一次，故初始化加完整走訪總Θ(n)，而非n次最壞O(h)必然等於Θ(nh)。若僅作不連續的任意查詢，不能把完整走訪的平均成本直接指派給每次查詢。若沒有parent、thread或顯式祖先堆疊，介面還須從root重搜祖先。

有序來源合併與線性建平衡樹

```
merge_unique(streams):
    H=min_heap of each nonempty stream's first record; pending=none
    while H not empty:
        (key,value,source)=H.pop_min()
        if pending==none: pending=(key,value)
        else if key==pending.key: pending.value+=value
        else: emit pending; pending=(key,value)
        advance source; if nonempty: H.push(next record from source)
    if pending!=none: emit pending
build_sorted(count):
    if count==0: return null
    left_count=count//2
    left=build_sorted(left_count)
    (k,v)=next(unique_sorted_stream)
    x=Node(k,v,left=left,right=null,parent=null,size=count)
    if left!=null: left.parent=x
    x.right=build_sorted(count-left_count-1)
    if x.right!=null: x.right.parent=x
    return x
```

有序來源合併與線性建平衡樹 （續）

先固定同鍵政策，例如value為出現次數、同鍵相加。三個來源[2,7,17]、[5,7,13]、[11,19]合併8筆，消去重複鍵後得到7筆2,5,7,11,13,17,19，其中7的value=2。k個來源、總輸入N筆時，heap合併O(k+N log(k+1))（含檢查k個來源的入口成本），k為常數時線性；不能對任意大k仍稱O(N)。不同鍵數U需先知道：可重讀來源時先計數再重播；一次性來源則先存U筆緩衝，不能憑空知道U。

build_sorted(7)先建左3筆：以5為根、2與7為子；下一筆11成全根，再建右3筆17(13,19)。每個節點在中序時恰消耗一筆，所以輸出中序等於輸入有序序列，歸納即證全子樹BST範圍。左右節點數相差至多1，遞迴高度O(log(U+1))，建樹$T(U)=T(\lfloor U/2\rfloor)+T(U-\lfloor U/2\rfloor-1)+\Theta(1)=\Theta(U)$。節點輸出空間Θ(U)、額外堆疊O(log(U+1))。

反例是把已排序U鍵逐個插入沒有平衡機制的BST，形成單鏈，總比較$1+2+\cdots+(U-1)=\Theta(U^2)$。輸入已排序可以幫助直接建構，不會自動讓一般insert變快。從有序陣列取中點也可達相同線性建樹；此串流版說明不必具備隨機索引。

引線二元樹重用空指標

- 以標記區分 child link 與 thread（引線），把原本空的左、右位置用來指向中序前驅或後繼。
- 找中序後繼時，若右連結是 thread，直接跟隨；若是真右子樹，先進右子，再沿真左子走到底。
- 完整走訪可不用遞迴或顯式堆疊，總時間 `O(n)`、額外走訪狀態 `O(1)`。
- 插入與刪除必須同步更新前後繼引線。若把 thread 當 child 遞迴，可能形成循環而不終止。

引線走訪與插入的具體追蹤

- 在原創 BST 中，葉 6 的左引線指向 3，右引線指向 8；葉 10 的左引線指向 8，右引線指向 12。
- 走到 6 後，右標記為 thread，故直接到 8。8 的右標記是真 child，先到 12，再沿真左 child 到 10。
- 若在 10 右側插入新鍵 11，原本該位置是指向 12 的 thread。新 11 的左、右引線分別指 10、12，再將 10 的右連結改為 child 並指向 11。
- 走訪結果改為 `1,3,6,8,10,11,12`。若仍把 10 的右標記當 thread，搜尋與走訪會採錯語義。資料指標與標記必須作為同一更新維護。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向引線的中序後繼與右插入

```
successor(x):
    if x.right_tag==thread: return x.right
    x=x.right
    while x.left_tag==child: x=x.left
    return x
insert_right(p,k):
    require p.right_tag==thread and k>p.key
    s=p.right
    require s==null or k<s.key
    x=Node(k,left=p,right=s,left_tag=thread,right_tag=thread)
    p.right=x; p.right_tag=child
    if s!=null and s.left_tag==thread and s.left==p: s.left=x
```

本章不使用header sentinel，最左前驅與最右後繼為null。從最左真child開始，不斷successor直到null。插11於10的右thread位置：原10.right=12且tag=thread；新11.left=10、11.right=12皆thread；改10.right=11且tag=child。12.left若是真child仍指10，不能任意改成11，否則會丟失真子樹；只有它原本是指10的前驅thread才更新。

某次successor可沿左鏈O(h)，但全程每條真子邊與引線只被有界次數使用，總O(n)，額外O(1)走訪狀態。更新時tag與pointer是共同表示；只改指標卻不改tag，會讓新11被當成thread跳過或被誤當遞迴child造成循環。與普通BST刪除共享抽象排序，但引線修復不能省略。

引線二元樹的刪除、端點與身分契約

採前節的雙向引線表示，無 header、最外側 thread 為 null，鍵互異。搜尋只沿標記為 child 的指標下降。先把兩子刪除化成後繼節點的至多一子刪除，再修前後繼及父連結；不能沿 thread 遞迴刪除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

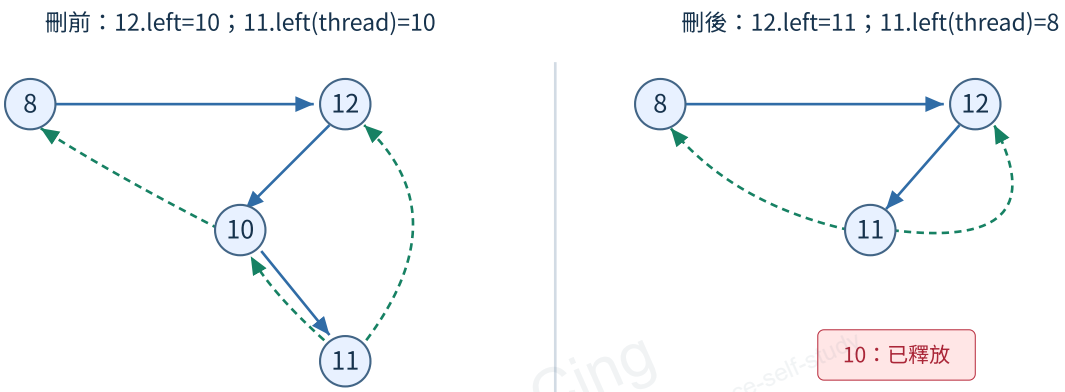
引線二元樹的刪除、端點與身分契約 （續）

```
predecessor(x):
    if x.left_tag==thread: return x.left
    x=x.left
    while x.right_tag==child: x=x.right
    return x
erase_threaded(k):
    p=null; z=root
    while z!=null and z.key!=k:
        if k<z.key:
            if z.left_tag==thread: return absent
            p=z; z=z.left
        else:
            if z.right_tag==thread: return absent
            p=z; z=z.right
    if z==null: return absent
    if z.left_tag==child and z.right_tag==child:
        p=z; s=z.right
        while s.left_tag==child: p=s; s=s.left
```

引線二元樹的刪除、端點與身分契約 （續）

```
        z.key_and_value=copy(s.key_and_value); z=s
    pred=predecessor(z); succ=successor(z)
    replacement=null
    if z.left_tag==child: replacement=z.left
    else if z.right_tag==child: replacement=z.right
    if pred!=null and pred.right_tag==thread and pred.right==z:
        pred.right=succ
    if succ!=null and succ.left_tag==thread and succ.left==z:
        succ.left=pred
    if p==null: root=replacement
    else if p.left_tag==child and p.left==z:
        if replacement!=null: p.left=replacement
        else: p.left=pred; p.left_tag=thread
    else:
        if replacement!=null: p.right=replacement
        else: p.right=succ; p.right_tag=thread
    release(z); return removed
```


引線二元樹的刪除、端點與身分契約 （續）



藍實線：真 child；綠虛線：thread。最外側 null thread 省略。
8.left 下的子樹 {1,3,6} 沒有改變，圖中省略。

具體刪除原樹中有右子 11 的 10：p=12、pred=8、succ=11、replacement=11。先把 11.left 的 thread 從 10 改指 8，12.left 再改真 child 11，11.right 的 thread 仍指 12，最後才釋放 10。完整中序由 [1,3,6,8,10,11,12] 變成 [1,3,6,8,11,12]，successor(8) 沿 12 的真左子到 11，predecessor(11) 直接跟 thread 到 8。

引線二元樹的刪除、端點與身分契約 （續）

刪葉 6 時，原父 3 的右 child 變成指向 successor=8 的 thread，不能保留 child 標記。刪唯一節點時 pred=succ=replacement=null，令 root=null 後釋放；空樹或找不到鍵不修改狀態。兩子分支複製完整 key-value 到目標位置，實際釋放的是後繼節點；此版只承諾字典值，不承諾未刪元素的節點 handle 穩定，複製成本與例外另計。

正確性分三層：後繼無真左子，故化簡後至多一子；移接 replacement 保留其他真 child 子樹及其中序順序；只有 pred 的後繼 thread、succ 的前驅 thread 及父的空側標記可能指向被刪節點，逐一修復後不留懸空 thread。所有 pointer 與 tag 一起維持，不能把真 child 無條件覆寫成 thread。搜尋、找後繼及前後驅各走至多高度 h 的路徑，總 O(h+1) 時間、O(1) 額外指標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小堆積有形狀與順序兩項不變量

- binary min-heap 是完全二元樹，且每個父鍵不大於子鍵。根因此是全域最小值。
- 堆積不要求左子小於右子，也不要求整棵左子樹小於右子樹，所以不能用 BST 搜尋方式找任意鍵。
- 合法陣列為 `[2,5,4,9,7,8]`。它不是整體排序，任意鍵搜尋最壞仍為 `O(n)`。
- Priority Queue ADT 支援 insert、find-min、delete-min。binary heap 是其中一種實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

sift-up 與 sift-down

- 插入 1，先放最後一格，再與父交換直到父鍵不大於它。本例從索引 6 經 2 到 0，結果 `[1,5,2,9,7,8,4]`。
- 刪最小值時，以末元素放根，再與較小子交換直到堆序恢復。只與任意一個子交換可能留下較小子在父下面。
- 沿更新路徑以外的邊未改變，路徑上的違規逐層修復，故保留堆序與完全性。
- insert、delete-min 的堆積重整最壞 `O(log n)`，find-min `O(1)`。若底層陣列擴容，某次 insert 另需線性搬移；該配置與搬移成本另以攤銷分析處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二元最小堆積的完整上浮與下沉

```
insert(v):
    A.append(v); i=len(A)-1
    while i>0:
        p=(i-1)//2
        if A[p]<=A[i]: break
        swap(A[p],A[i]); i=p
delete_min():
    require len(A)>0
    result=A[0]; last=A.pop()
    if len(A)==0: return result
    A[0]=last; sift_down(0,len(A)); return result
sift_down(i,n):
    while i<n//2:
        j=2*i+1
        if j+1<n and A[j+1]<A[j]: j+=1
        if A[i]<=A[j]: break
        swap(A[i],A[j]); i=j
build_heap():
    for i=n//2-1 down to 0: sift_down(i,n)
```

二元最小堆積的完整上浮與下沉 （續）

以本章[2,5,4,9,7,8]，插1後附加到6號、交換6↔2再2↔0，得[1,5,2,9,7,8,4]。刪最小時拿末4補根，得[4,5,2,9,7,8]，比較兩子5、2後換較小2，得[2,5,4,9,7,8]，下一子8不小於4而停止。單點heap刪除後直接回傳，不能在空A上寫A[0]。

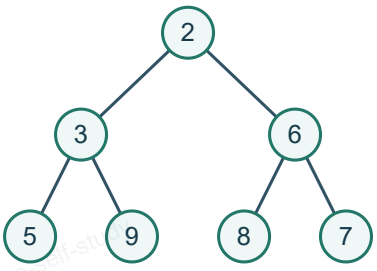
上述sift_down的迴圈條件 i<n//2（整數除法）恰好表示至少有左子，避免先算2i+1時溢位。n=3時只有i=0為內點；n=4時i=0、1都是內點。進入迴圈後才算j=2i+1並檢查j+1<n，避免缺右子時讀越界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建堆的每次比較與交換

自底向上建最小堆

節點與比較	交換後陣列（索引從 0 開始）
i=2 6 ≤ 7, 8	[9, 5, 6, 2, 3, 8, 7]
i=1 5 ↔ 2	[9, 2, 6, 5, 3, 8, 7]
i=0 9 ↔ 2	[2, 9, 6, 5, 3, 8, 7]
i=0 9 ↔ 3	[2, 3, 6, 5, 9, 8, 7]



i=2 的子堆已有效；每次下沉前，兩個子樹均為堆。

最後一列是完整陣列，右圖是同一狀態的樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自底向上建堆為何線性

- 從最後內部節點 $\lfloor n/2 \rfloor - 1$ 到根依序 sift-down。開始處理某節點時，其左右子樹已是堆。
- 高度為 h 的節點數為 $O(n/2^h)$，其下沉成本 $O(h)$。此寫法包含取整常數，不把近似節點數當成精確上界。
- 總成本 $O(n \sum_{h \geq 0} h/2^h) = O(n)$，而讀取輸入需 $\Omega(n)$，所以建堆為 $\Theta(n)$。
- 不能以「有 n 個點，每個 O(log n)」就宣稱緊界 $\Theta(n \log n)$。該乘法僅提供較鬆上界。

`std::priority_queue<int>` 預設取最大值。最小堆可指定 `std::greater<int>`，應確認比較方向與 min-heap 定義一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

怎麼選擇樹的表示？

結構	必須維護的條件	適合的操作
一般二元樹	左右子位置有意義	表達運算式或遞迴結構
BST	左子樹鍵 < 根鍵 < 右子樹鍵	搜尋、範圍與有序走訪；成本與高度有關
min-heap	完全形狀、父鍵 ≤ 子鍵	取最小值、插入和刪除最小值
雙向引線樹	child / thread 標記與指標一致	不用外部堆疊的中序走訪

先確認介面要保證什麼，再選表示。min-heap 的局部父子順序不能用來剪除整個子樹，因此任意鍵查找一般仍要線性時間。BST 若不維持平衡，搜尋也可能退化成線性。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；ISO C++ 草案 priority_queue。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study